Beta Guide

Beta Guide

This guide is the short path for validating a `gemstone-js` beta from an installed package or a clean checkout.

Install Shape

For local TypeScript development:

```
npm install gemstone-js
```

For real GemStone/S access from Node, install the optional native package too:

```
npm install gemstone-js @gemstone-js/native
```

The JavaScript package keeps `@gemstone-js/native` optional so browser-adjacent tooling, docs builds, and mock-runtime tests can install without a GemStone GCI client library. Production Node deployments that connect to a Stone should pin both package versions together.

First Check

Set the connection environment expected by `Session.configFromEnv()`:

```
export GS_STONE=gs64stone
export GS_NETLDI=netldi
export GS_HOST=localhost
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
```

Then run the setup check:

```
gemstone-js-doctor
gemstone-js-doctor --live
```

Existing GemStone bridge shells can keep using `GS_USER`, `GS_PASS`, `GS_NETLDI_HOST`, `GS_NETLDI_NAME_OR_PORT`, and `GS_SERVICE`; the canonical JavaScript names above win if both are set.

Native Backends

Node uses the raw `@gemstone-js/native` `Gci` binding by default. Enable the worker backend when you want calls for each session queued through the native package's dedicated session worker:

```
export GS_NATIVE_SESSION_WORKER=1
gemstone-js-doctor
```

Application code can also pass `nativeSessionWorker: true` to `Session.connect()`. Worker startup is intentionally strict: if the installed native package is missing `createGciSessionWorker()` or one of the methods the runtime needs, setup fails instead of silently falling back to the raw backend.

Use the raw backend while isolating low-level GCI or native library failures. Use the worker backend for beta production trials where blocking native calls should not run on the main JavaScript thread.

Live Smoke

From a checkout, the local verify command is still the default release gate:

```
npm run verify
```

Real Stone coverage is opt-in:

```
GS_RUN_LIVE=1 npm run test:live
GS_RUN_LIVE=1 GS_NATIVE_SESSION_WORKER=1 npm run test:live
npm run test:live:worker
```

The live path covers connection setup, selector sends, arrays, dictionaries, globals, persistent roots, GStore, generated wrappers, larger query paths, request scopes, framework adapters, pool pressure, and native worker stress.

Generated Wrappers

Generated wrappers are expected to be committed and reviewed. Keep the manifest and decorator paths checked with:

```
npm run codegen:check
npm run codegen:scan:check
```

Use `examples/codegen.manifest.json` for manifest-driven wrappers and `examples/booking.decorators.ts` for decorator scanning. The generated examples cover typed arguments, array and dictionary argument marshalling, value returns, raw OOP returns, and retained typed-object returns.

The current object-mapping support is explicit: use `TypedOop<T>`, `GsDict`, `PersistentRoot`, generated selector wrappers, and the opt-in `mappedObject()` helper for async property-style methods, relationship handles, root/global entry points, retained-handle lifetime control, and bounded snapshots. Use `transparentObject()` when you want higher-transparency `await booking.status` reads, queued assignment writes, optional local caching, and request-scoped proxy identity. The object-mapping manifest schema and `examples/object-mapping.manifest.json` are committed for reviewable mapping metadata; the generated `*Ref` classes, repository helper generator, and Explorer/VS Code mapping views remain outside the beta support boundary.

Installed Package Proof

Before publishing or trialing a beta from tarballs:

```
npm run release:check
npm run native-install:check
npm run api-contract:installed
```

The native install check packs `gemstone-js` and the sibling `../gemstone-js-native` checkout, validates npm pack integrity metadata, checks exact optional dependency version parity, inspects the native tarball for worker files and a `.node` binary, verifies the installed dependency graph, and probes the installed worker backend with `gemstone-js-doctor`.

The full release-candidate gate is:

```
GS_RUN_LIVE=1 npm run release-candidate:check
```

That runs local verification without inheriting `GS_RUN_LIVE`, then reruns the native install check with worker-mode live smoke from the temporary installed package and finishes with the checkout worker live regression.

Support Boundary

For the beta, the supported path is Node with the packaged TypeScript API, checked-in generated wrappers, the mock runtime for local tests, and `@gemstone-js/native` for live GemStone/S access.

Deno and Bun FFI adapters are scaffolded and useful for continued porting work, but they should be treated as experimental until live platform coverage catches up. Visual tooling and a VS Code extension should remain out of the beta critical path until the package, native worker backend, and release workflow settle. `smalltalkBridge()`

and `transparentObject()` are supported opt-in mapping conveniences for tools and application code that accept explicit async remote calls. Generated connector-style mapping is still a good next product-polish track, not a beta-critical runtime contract.

Troubleshooting

- If `gemstone-js-doctor` cannot import `@gemstone-js/native`, install the optional native package or run local checks with `--no-native`.
- If worker mode reports a missing method surface, update `@gemstone-js/native` to the matching beta version or unset `GS_NATIVE_SESSION_WORKER`.
- If live login fails, verify `GS_STONE`, `GS_NETLDI`, `GS_HOST`, `GS_USERNAME`, and `GS_PASSWORD`, then run `gemstone-js-doctor --live --json` to inspect non-secret diagnostics.
- If package behavior differs from the checkout, run `npm run native-install:check` to exercise the tarball-installed layout.